

QA|WARE
SOFTWARE ENGINEERING

Chapter 3: Virtualization

Lukas Buchner
lukas.buchner@qaware.de

Requirements Cloud Computing



- **Organization of Resources** in a data center
- More efficient usage of given resources to **minimize costs**
- **Isolation of resources**. Customers should not see and influence others. Avoid side effects. Security
- **Decoupling from the hardware** for more operational flexibility and robustness in the event of failures
- Resources should be allocated flexibly. Control using **software defined resources**

- The solution to these requirements is **virtualization**, which makes **cloud computing** possible in the first place..

Virtualization

Virtualization: the creation of virtual realities and their mapping onto physical reality.

<u>Purpose:</u>

- **Multiplicity:** Creation of multiple virtual realities within a single physical reality
- **Decoupling:** Dissolve the bond and dependency on reality
- **Isolation:** Avoiding physical side effects between virtual realities



Virtualization types

Virtualization is representative of several fundamentally different concepts and technologies.

Virtualization of hardware infrastructure

1. Emulation
2. Full virtualization (Type 2 virtualization)
3. Para virtualization (Type 1 virtualization)

Virtualization of software infrastructure

4. Operating system virtualization (*Containerization*)
5. Application virtualization (*Runtime*)

Virtualization and Cloud Computing

- **Decoupling from the hardware** for more operational flexibility and robustness in the event of failures.

- **Standardization of resource capacities** on heterogeneous and changing hardware (“S instance”, “XL instance”).

- **Centralized control and provision of computing resources** about the software-defined resources provided by virtualization.



Virtualization types: Hardware virtualization

What is being virtualized?



- **Processor**
  - Virtual cores
  - Dispatching processor instructions to real cores
- **Memory**
  - Virtual main memory partition
  - Management of the real representation (in RAM, on hard disk, ballooning)
- **Network**
  - Virtual network interfaces and virtual network infrastructures (VLAN)
  - Bridges between virtual and real networks
- **Storage**
  - Virtual hard disk drives. Mapping to files in the real file system. Volumes either pre-allocated or dynamically growing.
  - Virtual SANs (Storage Area Networks) via distribution of a virtual disk drive data to many storage units.

What is being virtualized?



- **GPU**
  - Frame buffer (2D-Array with pixel data)
  - 3D-Functionality (DirectX, OpenGL), see https://en.wikipedia.org/wiki/GPU_virtualization
  - Computing (AI, Simulations) (Increasingly important for the Cloud)
- **Peripherals such as USB, mouse, keyboard**
- **Timer, Interrupt Controller**

Classic structure of an operating system with computer architecture support

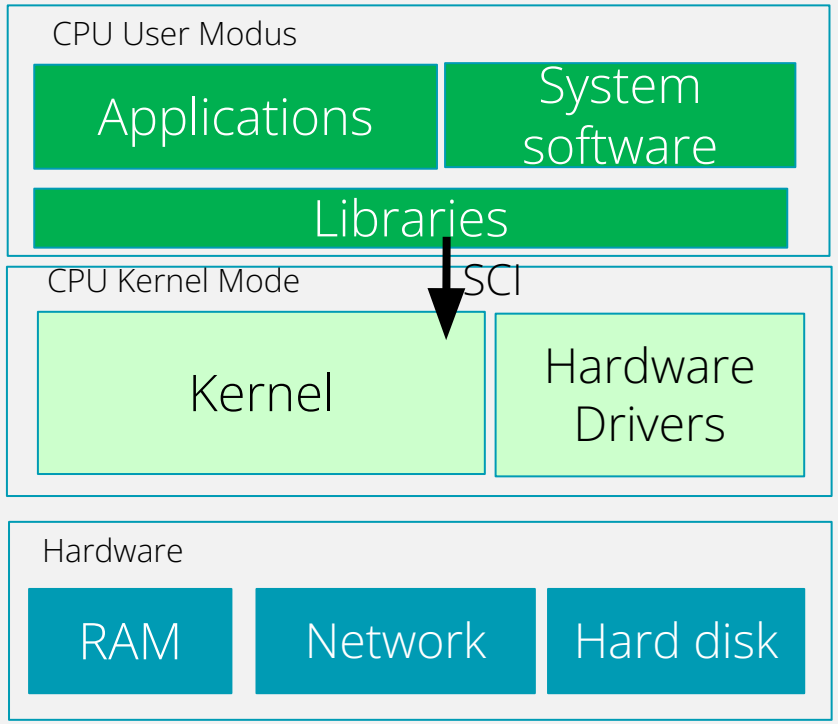
QA|WARE

CPU User Mode

- Lowest authorization level
- No direct hardware access
- Memory protection via the memory management unit

CPU Kernel Mode

- User Mode calls the kernel via the System Call Interface (SCI).
- Currently, the SCI in Linux consists of about 380 system calls.
- Highest level of authorization
- Privileged CPU instructions
- Access to hardware via drivers
- Takes over, for example, file system management and application scheduling



Hardware support

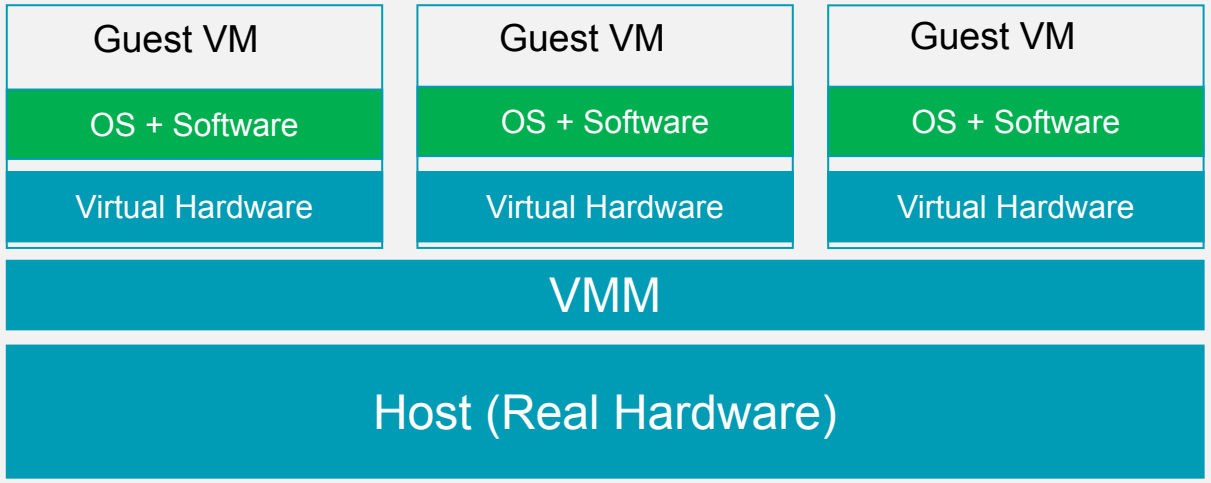


- **Software based virtualization**
  - In the classic operating model, only the user mode has the necessary isolation properties. The kernel mode can be emulated here, e.g. using “trap-and-emulate”, with at least a 10% performance loss.
- **Hardware assisted virtualization**
  - CPU extensions such as Intel-VT and AMD-V were therefore often developed. These extensions add a new processor mode (e.g. virtual execution mode) in which the guest operating system perceives itself as working with full privileges, but the host operating system remains protected.
- Virtual main memory partition in real physical memory. (The zero is shifting). Management of the real representation using the management memory unit (MMU).
- For the passthrough of the interfaces of real hardware devices, the displacement of the zero must be compensated by an IOMMU (I/O Memory Management Unit).

Hardware virtualization

- **Hardware virtualization** divides the resources of a computer system so that they can be used by multiple independent operating system instances.

- The requirements of the operating system instances are intercepted by the virtualization software (**virtual machine monitor, VMM**) and implemented on the actual hardware.



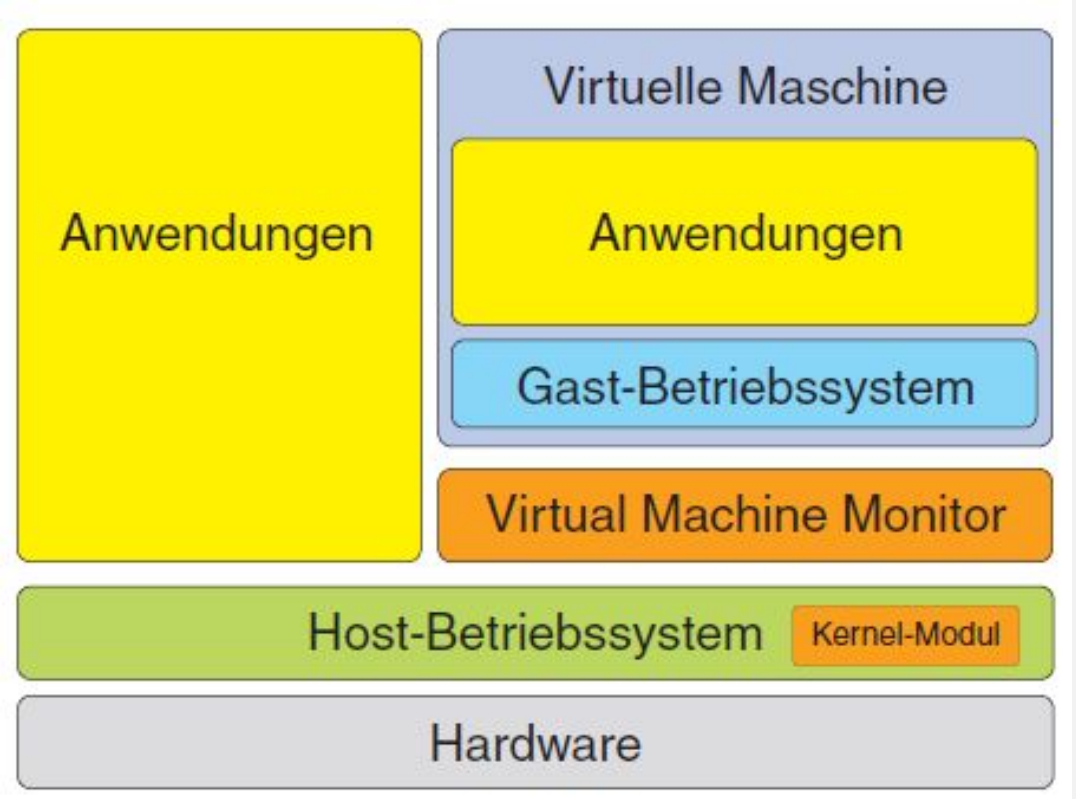
Host
- The computer that runs one or more virtual machines and provides the necessary hardware resources.

Guest
- A runnable / running virtual machine

VMM (Virtual Machine Monitor)
- The control software for managing guests and host resources

Hardware virtualization: Full virtualization

- Each guest operating system has its own virtual computer with virtual resources such as CPU, main memory, disk drives, network cards, etc.
- The VMM runs as an application hosted by the host operating system (type 2 hypervisor)
- The VMM distributes the computer's hardware resources among the VMs
- The VMM partially emulates hardware that is not designed for simultaneous access by multiple operating systems (e.g. network cards, graphics cards)
- Performance loss: 5-10%.



Hardware virtualization: para-virtualization

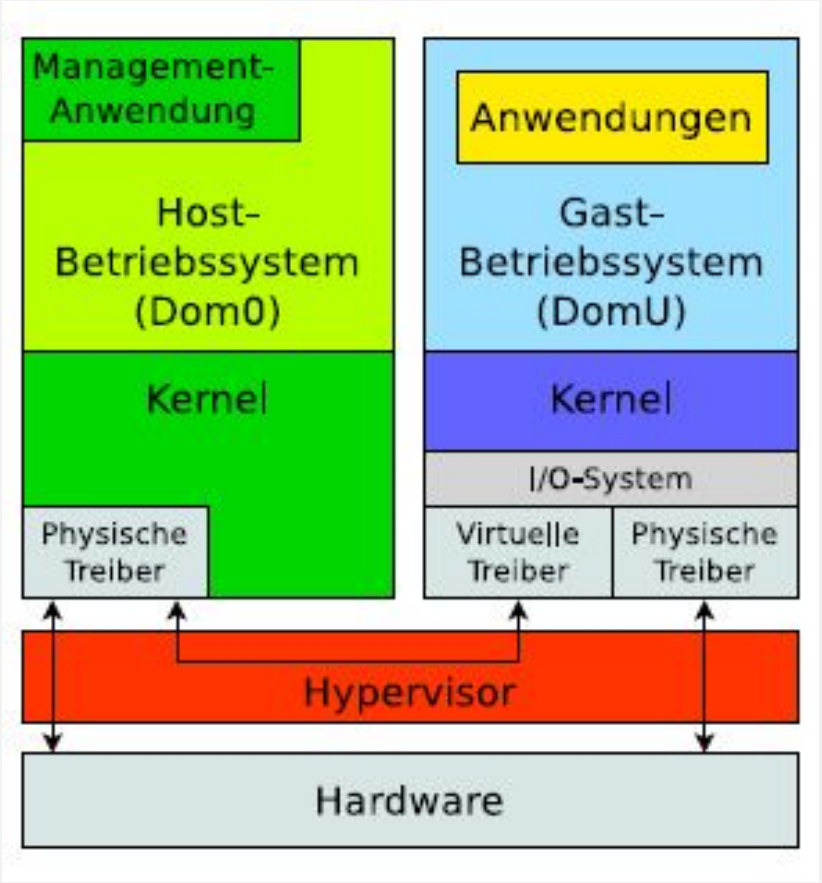
The hypervisor runs directly on the available hardware. It is therefore an operating system that is exclusively designed for virtualization.

Virtual drivers must be added to the guest operating system in order for it to interact with the hypervisor.

No directly low-level virtualized hardware resources (CPU, RAM, etc.) are available to the guest operating system, but an API is available for use by the virtual drivers.

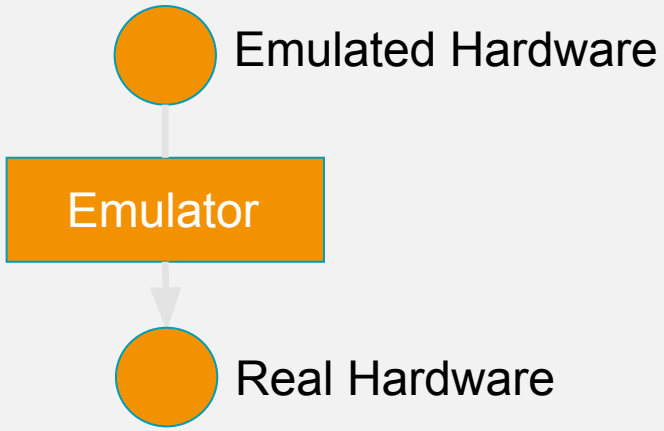
- Supported operating systems and hardware variants from the guest's point of view are limited per hypervisor implementation.
- The hypervisor uses the drivers of a host operating system to access the real hardware. This eliminates the need for the hypervisor to implement its own drivers.

Performance loss: 2-3%



For completeness:
What is emulation and application virtualization?

- Emulation: Recreates the hardware of a non-existent or incompatible computer system or parts of a corresponding computer system.
- Purpose, among other things: to preserve old software.
- (Example: Gameboy in the browser)



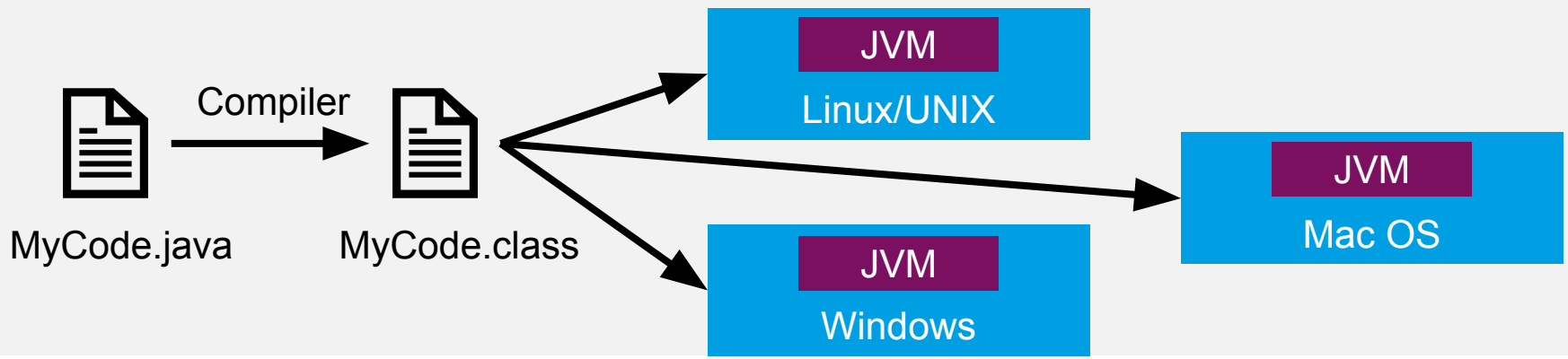
Virtualization, but high-performance

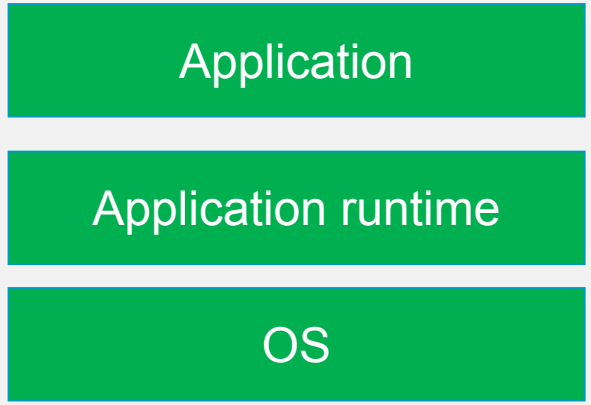


- Emulations fulfill many of the requirements for cloud computing, such as isolation and decoupling. However, they are very slow when replicating an entire computer architecture and are therefore unsuitable for mass productive use.
  - The main culprit is the **CPU**.
- Can the same goals be achieved with minimal additional resource expenditure?
  - Answer: Yes, but only if the guest architecture of the virtualized system is the same as the host architecture.
  - x86 host → x86 guest

For completeness: What is emulation and application virtualization?

- Application virtualization: provides applications with a programming interface and a runtime environment that is completely decoupled from the underlying operating system.
- Purpose, among other things: portable applications.
- Beispiel: JVM





Virtualization in the enterprise environment



In addition to the advantages mentioned so far, today's VM solutions offer many more features:

- **Resource management during operation**
  - Memory ballooning: dynamically increasing main memory
  - Changing the number of virtual cores
  - Changing disk size with virtual SANs (Storage Area Networks)
- **Live Migration**: Moving the running physical machine to a different hardware within milliseconds
  - CPU State
  - Memory
  - Storage
  - Network
- Attention: Creating (genuine) randomness is even more difficult in a VM than with real hardware (e.g. using mouse and keyboard input).
- This is where hypervisors provide interfaces to obtain additional randomness.



Hardware virtualization with Vagrant and VirtualBox

Hardware virtualization with Vagrant and VirtualBox



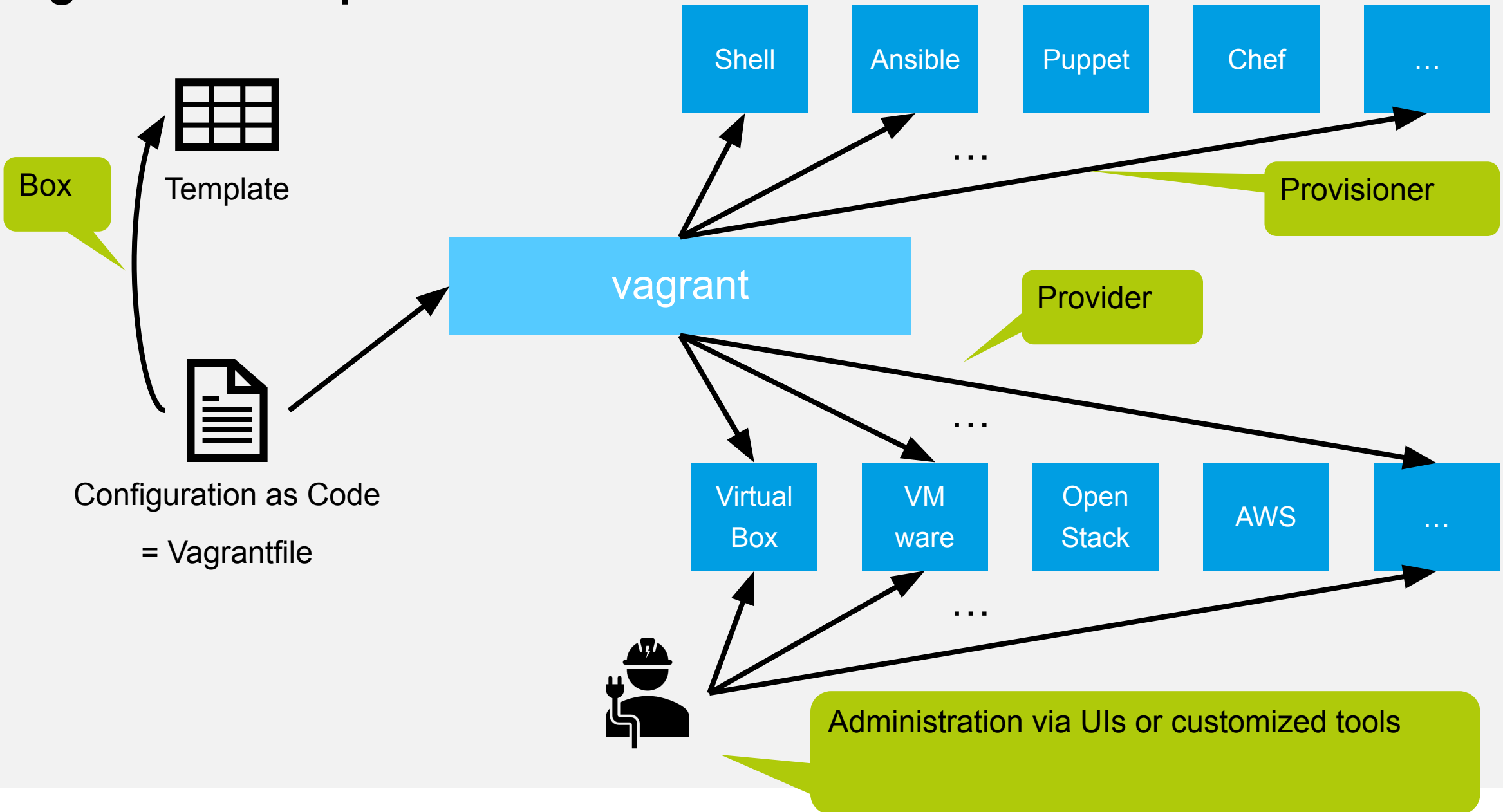
+



Open Source Type 2 full virtualization software for Windows, Linux, MacOS and Solaris.

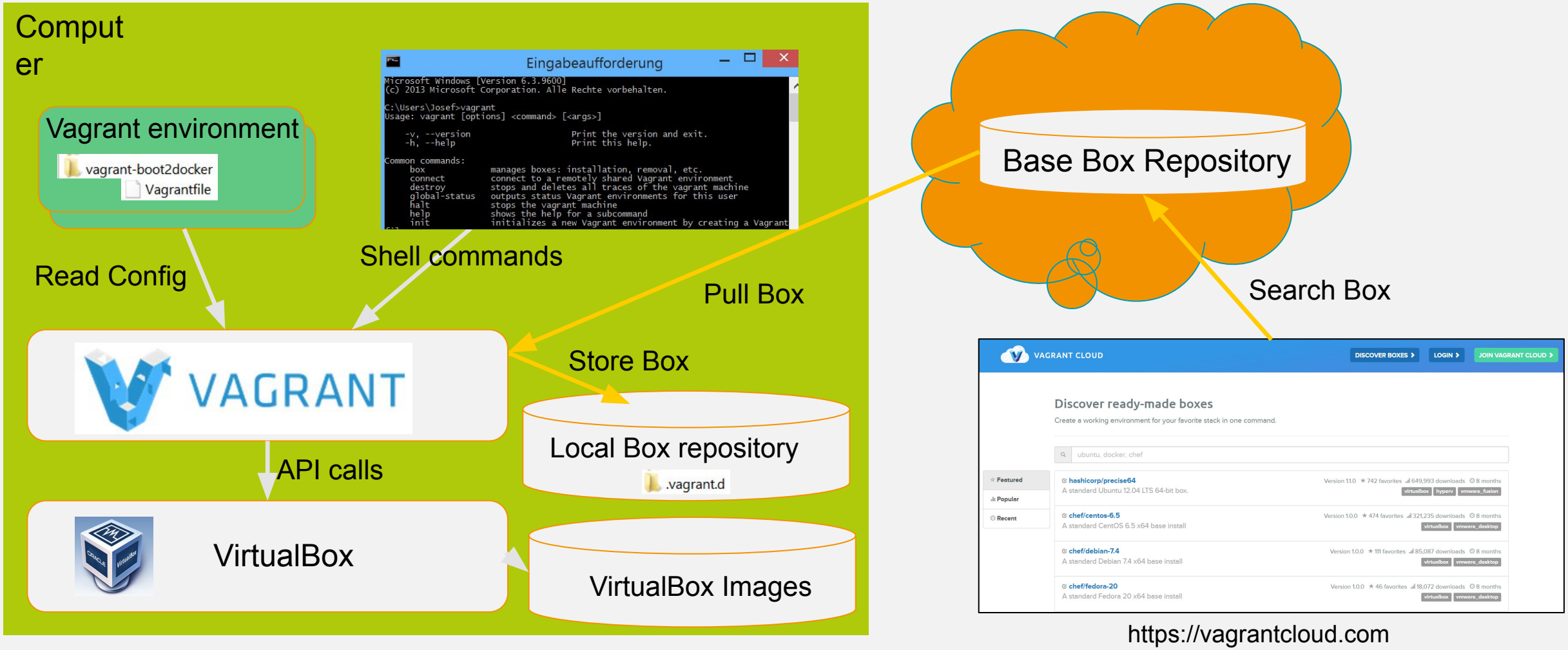
Automation software for virtual environments on a computer. Create and control virtual machines via command line.

Vagrant: Concepts



Vagrant: A schematic overview.

QA|WARE



https://vagrantcloud.com

The Vagrantfile describes the virtual machine to be created.

```
# -*- mode: ruby -*-
# vi: set ft=ruby :

# Vagrantfile API/syntax version. Don't touch unless you know what you're doing!
VAGRANTFILE_API_VERSION = "2"

Vagrant.configure(VAGRANTFILE_API_VERSION) do |config|
        # My base box
        config.vm.box = "chef/ubuntu-14.04"

        # Define shell provisioning
        config.vm.provision :shell, path: "bootstrap.sh"

        # Define docker provisioning
        config.vm.provision "docker" do |d|
                d.run "nginx1", image: "dockerfile/nginx", args: "-p 8080:80", daemonize: true
                d.run "nginx2", image: "dockerfile/nginx", args: "-p 9080:80", daemonize: true
                d.run "haproxy", image: "dockerfile/haproxy", args: "-p 80:80 --link nginx1:nginx1 --link nginx2:nginx2 -v /vagrant:/haproxy-override"
        end

        # Configure VirtualBox
        config.vm.provider "virtualbox" do |v|
                v.memory = 1024
                v.cpus = 4
        end

        # Forward ports
        config.vm.network :forwarded_port, host: 80, guest: 80
        config.vm.network :forwarded_port, host: 8080, guest: 8080
        config.vm.network :forwarded_port, host: 9080, guest: 9080
end
```

Vagrantfiles are written in Ruby.

Definition of the base box

Configuration of provisioning

Configuration of the virtualization provider

Configuring the network

A typical workflow with Vagrant.

#	Shell commands	Description
1	`mkdir <box-dir>` `cd <box-dir>`	Create a directory for the Vagrant environment and switch to it

2	`vagrant init`
	`[<box-name>]`
	`[<box-url>]`
		Initialize a Vagrant environment. This first creates a Vagrantfile and initializes it with the name and URL of the box (if specified).

3		Customize the Vagrantfile as needed (e.g. assign an IP, port mapping between host and guest, directory share between host and guest, etc.)
4	`vagrant up`	Start the virtual machine (Box □ virtual machine) and configure it according to the Vagrantfile
5	`vagrant ssh`	Connect to the virtual machine via SSH
6	`exit`	Exit the SSH command line in the virtual machine
7	`vagrant halt`	Stop the virtual machine.

Other useful commands:

- reload: Restarts a VM and updates the configuration according to the Vagrantfile
- package: Recreates a box from a virtual machine

Further commands: http://docs.vagrantup.com/v2/cli/index.html

Vagrant commands on the command line



- vagrant box add – allows you to install a box (or VM) to the local machine
- vagrant box remove – removes a box from the local machine
- vagrant box list – lists the locally installed Vagrant boxes
- vagrant init – initializes a project to use Vagrant
- vagrant up – starts up the vagrant VM
- vagrant suspend – saves the state of the current VM.
- vagrant resume – will load up the suspended VM.
- vagrant halt – will shut down the VM, saving configuration. (restart with 'up' command)
- vagrant destroy – will destroy the VM with all config changes.
- vagrant reload – apply Vagrant configuration changes (like port forwarding) without rebuilding the VM.
- vagrant status – tells you the current state of the Vagrant project's VM
- vagrant gem – install Vagrant plugins via RubyGems
- vagrant ssh – short cut to SSH into the running VM
- vagrant package – create a distribution of the VM you have running.
- vagrant <command> –help - Command that will provide man pages for a vagrant command.

https://developer.hashicorp.com/vagrant/docs/cli



Types of virtualization: Operating system virtualization

Hardware virtualizers are heavyweights



- Each **VM** includes a virtual copy of a complete operating system and requires significant **RAM** and **CPU** resources, which are difficult to change dynamically.
- Software development with **VMs** is slower and more complex.
- Due to the size of the **images**, portability is an issue.
- There is no compatibility with other **VM** solutions. Moving between data centers is not easily possible.

Linux Kernel Namespaces (Isolation through visibility)

A feature of the **Linux kernel** that restricts the view and access to the system:

- process space / process IDs
- network interfaces
- host name
- file system mounts
- IPC (Inter-Process Communication)
- user accounts

The restrictions are transparent to the isolated process.
Namespaces can be nested.

See https://success.docker.com/KBase/Introduction_to_User_Namespaces_in_Docker_Engine

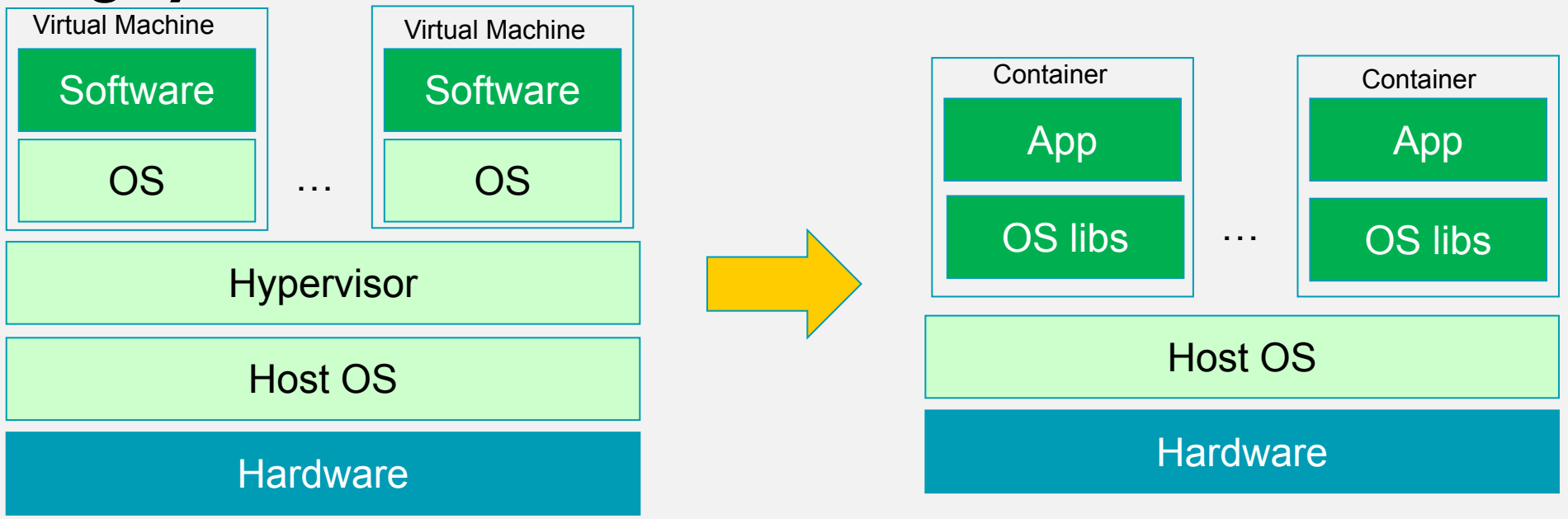
Linux Cgroups (Isolation through boundaries) /1

A feature of the **Linux kernel** that was largely developed by Google

Grouping **processes** into communities with defined and restricted resource access to:

- Processor
- Main memory
- I/O (esp. network)
- Disk

Operating system virtualization



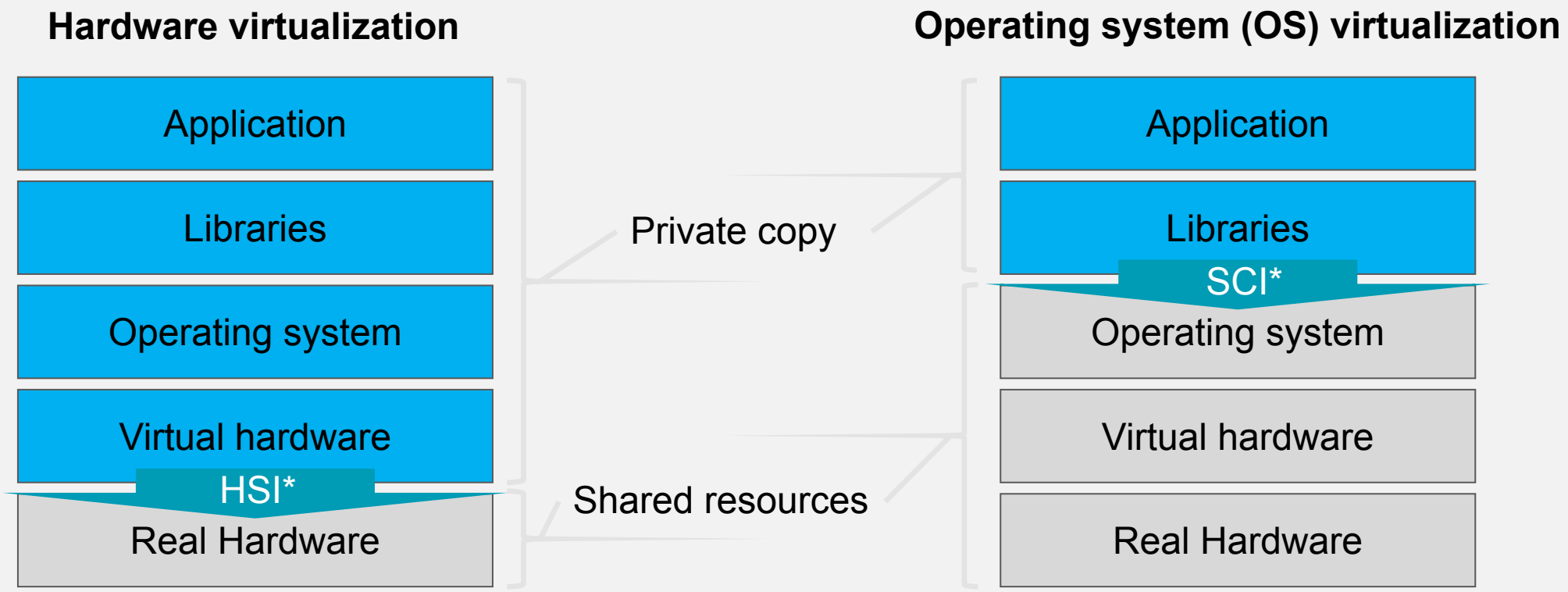
Lightweight virtualization approach: There is no hypervisor. Each app runs directly as a process in the host operating system. However, this is maximally isolated by corresponding OS mechanisms (e.g. Linux LXC).

- Isolation of the process through kernel namespaces (regarding CPU, RAM and disk I/O) and containments
- Isolated file system
- Separate network interface

CPU/RAM overhead generally not measurable (~ 0%)

Startup time = start duration for the first process

Hardware- vs. Operating system virtualization



- Better insulation
- Higher security

- Smaller private copy volume
- Lower overhead
- Faster startup time

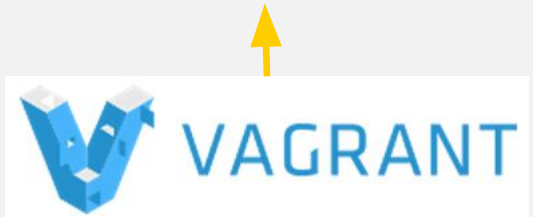
*) HSI = Hardware Software Interface
SCI = System Call Interface

Example technologies

QA|WARE

Hardware virtualization
Vagrant und VirtualBox





OS virtualization: Docker





podman

containerd



OS Virtualization with Docker

Containerization with Docker



http://www.srf.ch/kultur/im-fokus/brasilien/favelas-im-wandel-die-armen-muessen-weichen



Standard format for operations: start, stop, configure, wire, debug + software logistics.

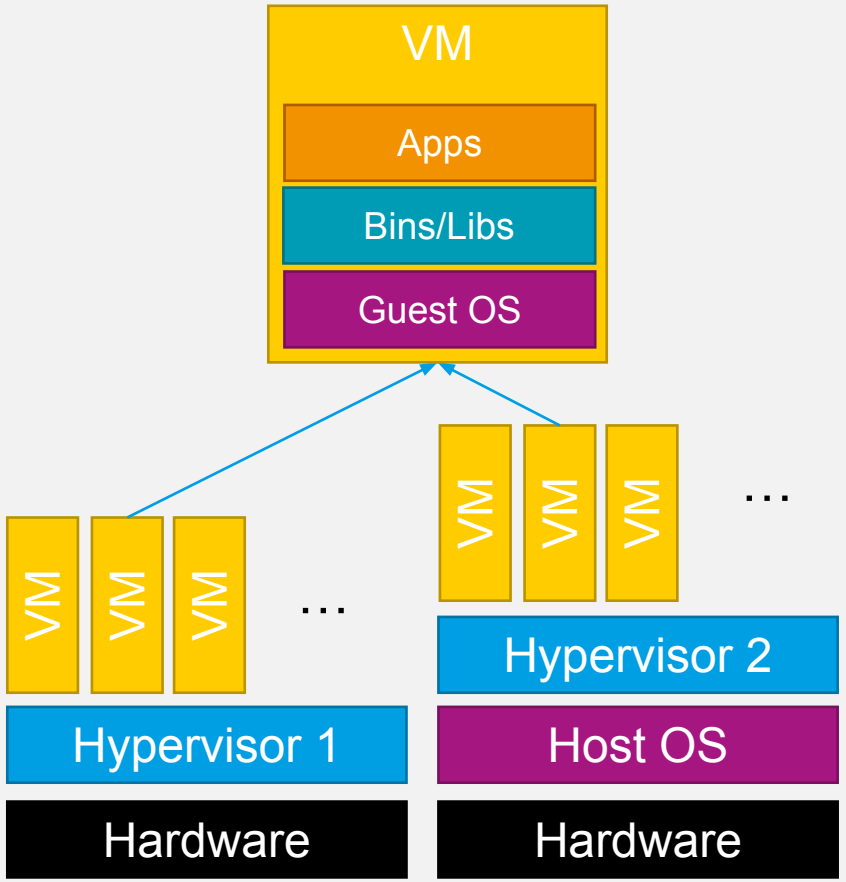
Docker

Docker is an automation environment for operating system virtualization.

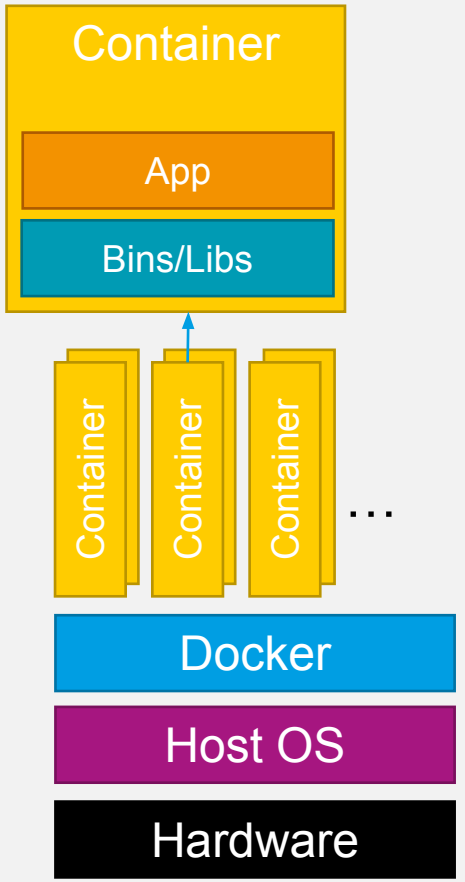
Docker currently supports Linux as a host operating system. Since Windows Server 2016, a Windows variant is now also available.

Docker was developed as a tool for a cloud provider and is now one of the most visible and active open-source ecosystems.

Containerization with Docker

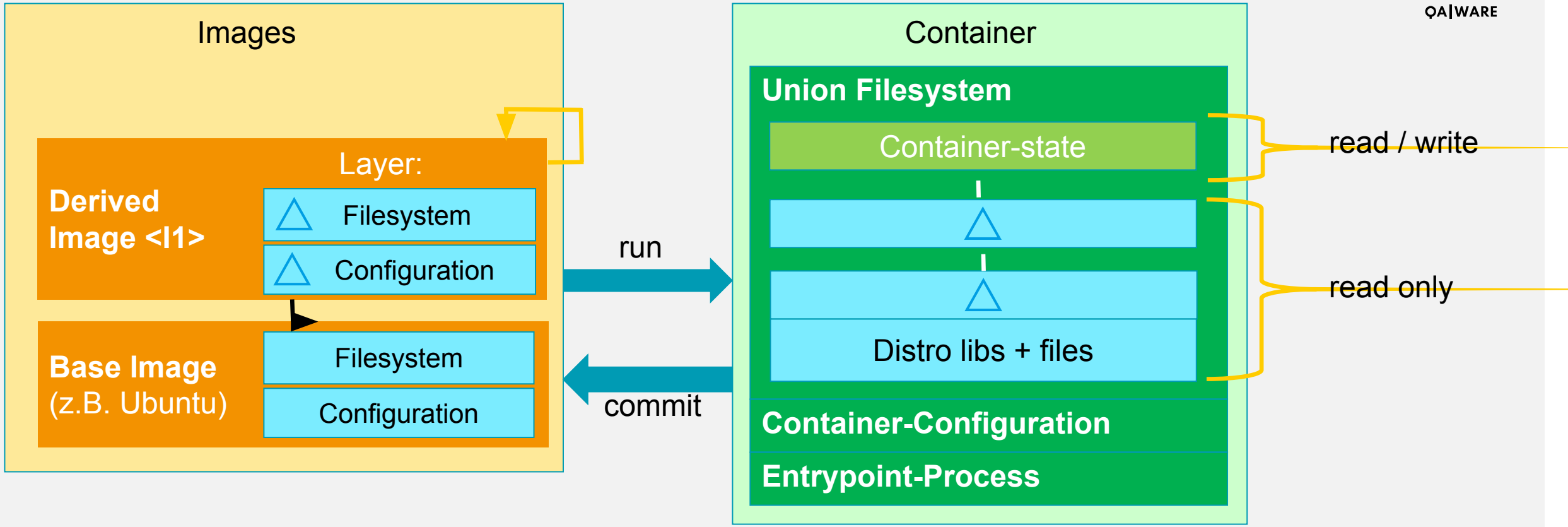


Type 1 / Type 2 Virtualization



Containerization

At the center of Docker are images and containers.



stationary and transportable state

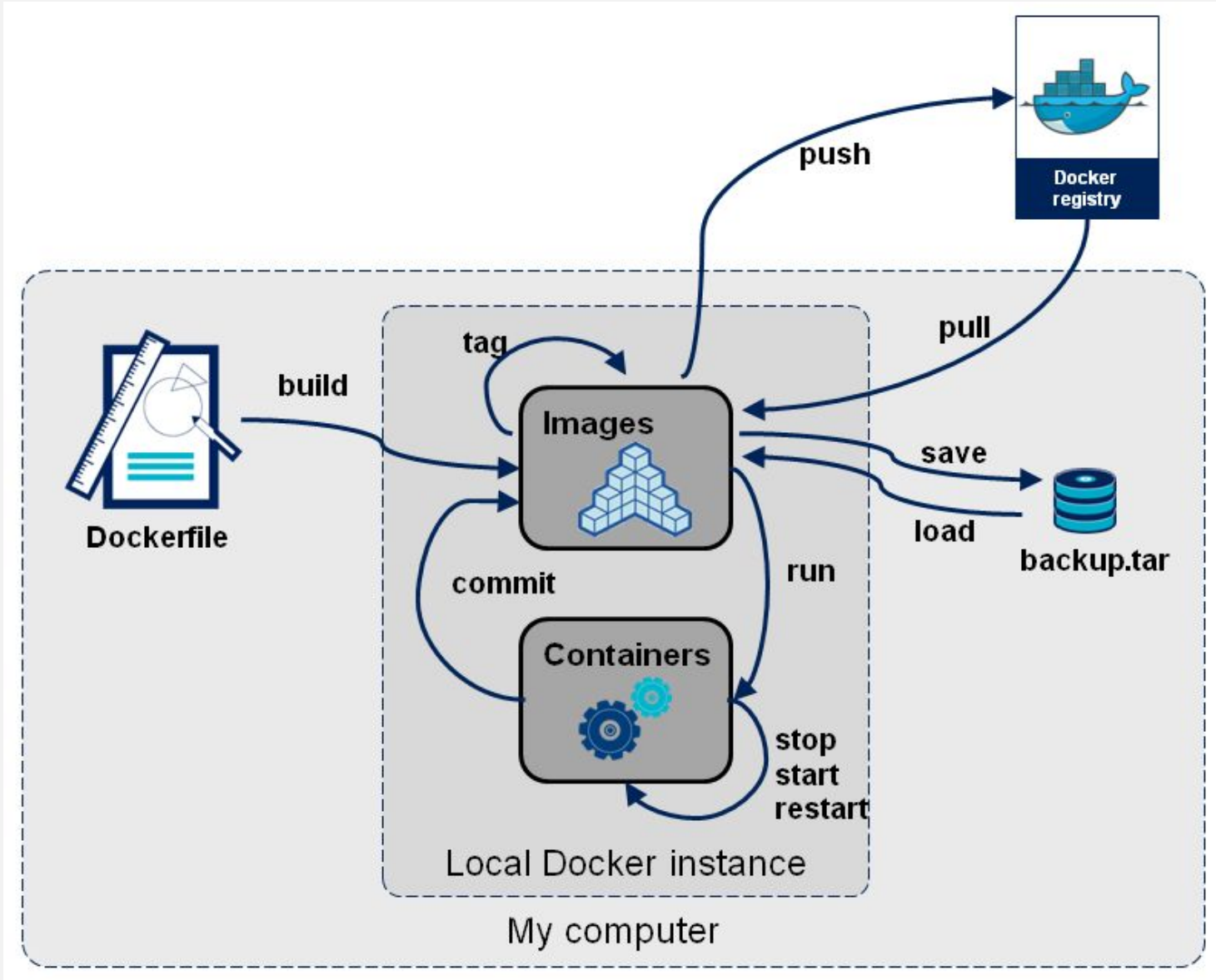
Running state

A container runs as long as its entrypoint process is in the foreground. Docker remembers the container state..

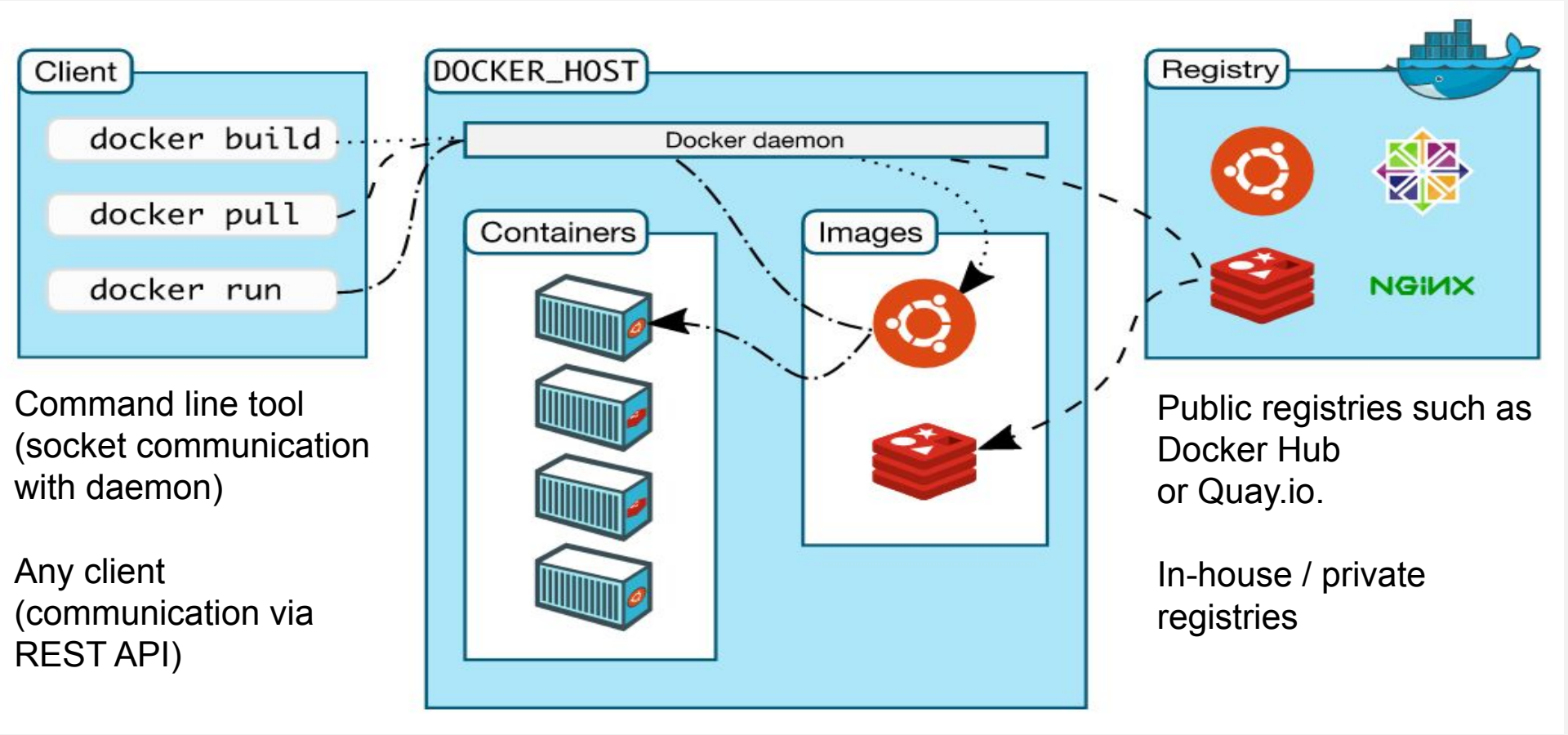


Demo

The Docker Workflow.



The Docker architecture.



Command line tool (socket communication with daemon)

Any client (communication via REST API)

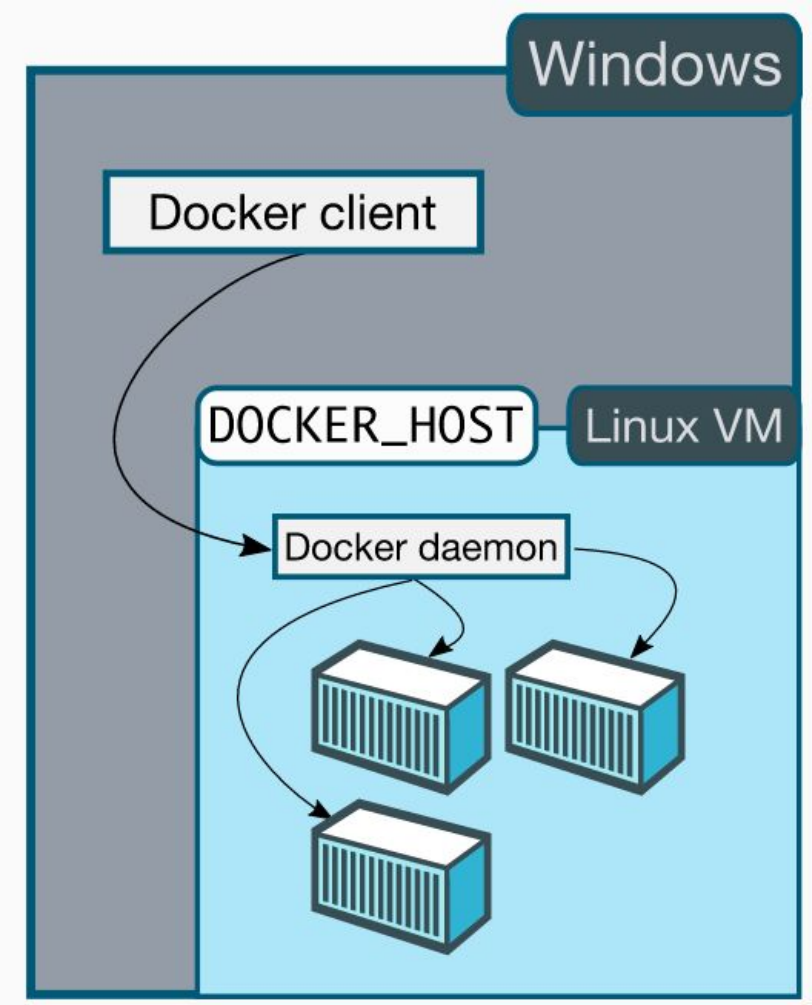
Public registries such as Docker Hub or Quay.io.

In-house / private registries

The docker daemon is the central control unit and runs directly as a process in the host operating system. It manages all local

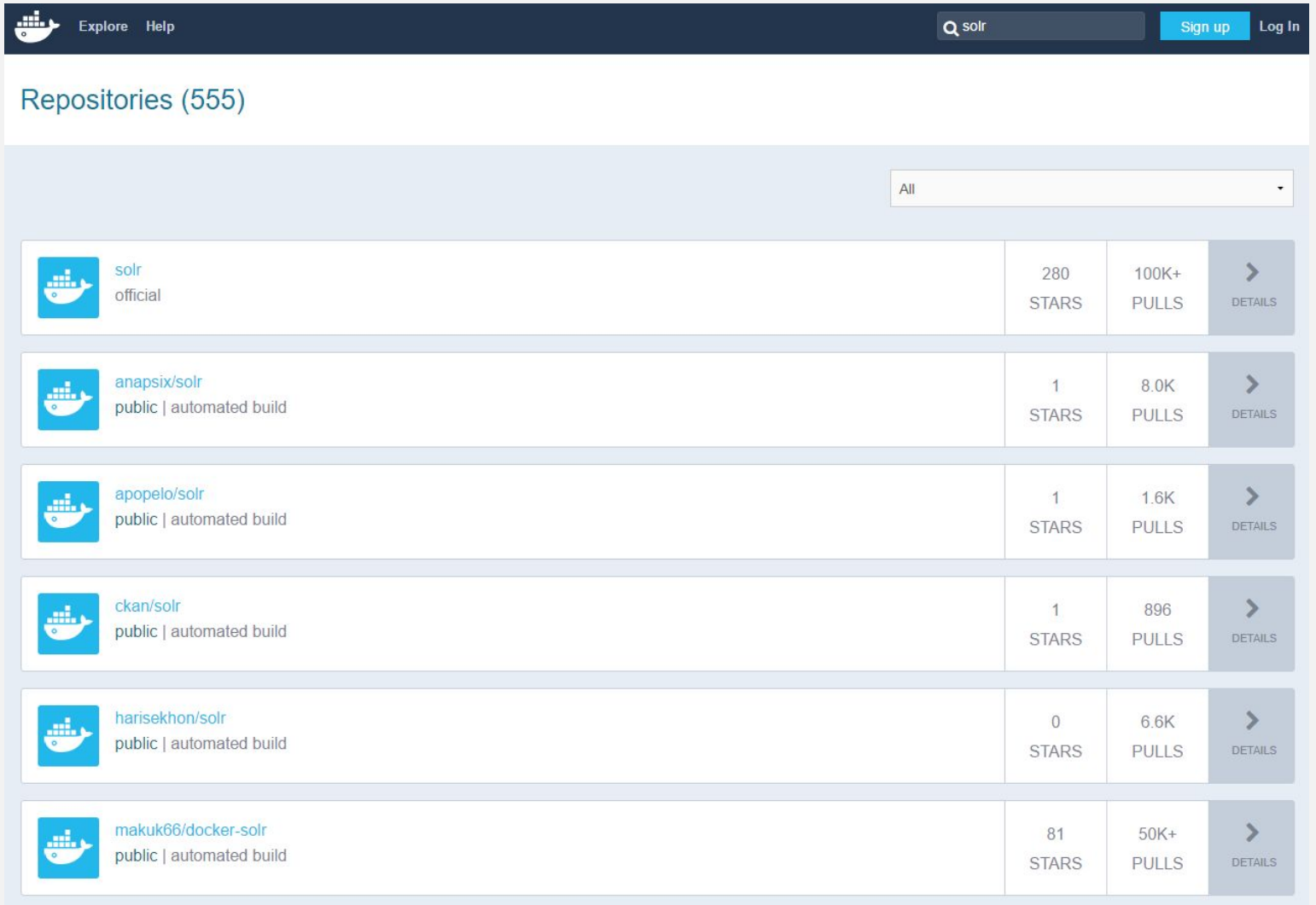
Bildquelle: https://docs.docker.com/engine/docker-overview/

Interaction between host and Docker machine.



Source: http://docs.docker.com/engine/installation/windows

hub.docker.com is the standard public registry for Docker images.



Provisioning Images with the Dockerfile



A Dockerfile generates a new image based on another image. This automates the following actions:

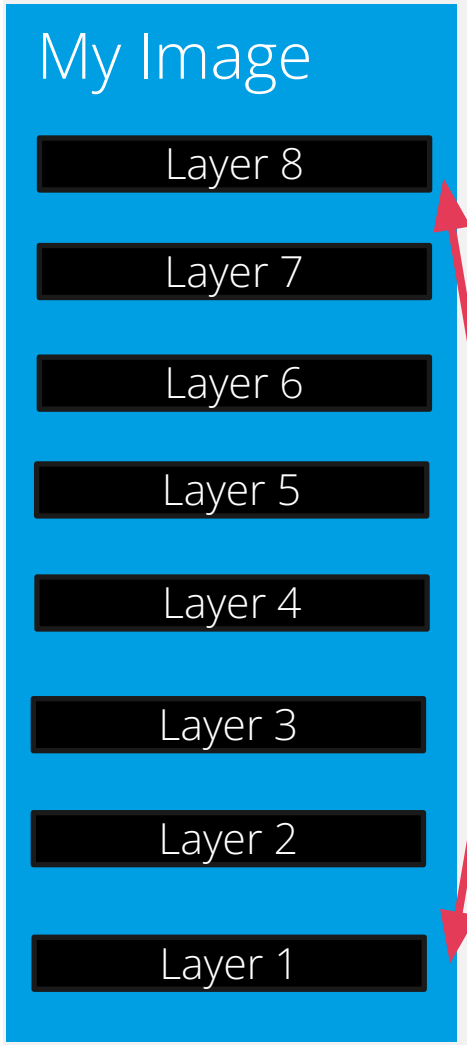
- Configuration of the image and the resulting containers
- Execution of provisioning actions

A Dockerfile is thus an image representation as an alternative to a physical image
(a building share vs. a building component).

- Repeatability in the construction of containers
- Automated creation of images without having to distribute them
- Flexibility in the configuration and in the software versions used
- Simple syntax and therefore easy to use

Command: `docker build -t <target_image_name> <Dockerfile>`

The Dockerfile defines Structure and content of the image.



Never use latest. Antipattern

```
FROM qaware/alpine-k8s-ibmjava8:8.0-3.10
LABEL maintainer="QAware GmbH <qaware-oss@qaware.de>"

RUN mkdir -p /app

COPY build/libs/zwitscher-service-1.0.1.jar /app/zwitscher-service.jar
COPY src/main/docker/zwitscher-service.conf /app/

ENV JAVA_OPTS -Xmx256m

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/zwitscher-service.jar"]
```

Dockerfile commands

Element	Meaning
FROM <image-name>	Sets to base image (where the new image is derived from)
MAINTAINER <author>	Document author
RUN <command>	Execute a shell command and commit the result as a new image layer (!)
ADD <src> <dest>	Copy a file into the containers. <src> can also be an URL. If <src> refers to a TAR-file, then this file automatically gets un-tared.
VOLUME <container-dir> <host-dir>	Mounts a host directory into the container.
ENV <key> <value>	Sets an environment variable. This environment variable can be overwritten at container start with the –e command line parameter of `docker run`.
ENTRYPOINT <command>	The process to be started at container startup
CMD <command>	Parameters to the entrypoint process if no parameters are passed with `docker run`
WORKDIR <dir>	Sets the working dir for all following commands
EXPOSE <port>	Informs Docker that a container listens on a specific port and this port should be exposed to other containers
USER <name>	Sets the user for all container commands

Typical commands of a Docker Workflows

Command	Action
`docker build -t <image> .`	Build Docker image with given tag in current directory
`docker images`	Prints all local images

`docker run`
`-d`
`-v <volume mounts>`
`-p <host-port>:<container-port>`
`<image> <entrypoint process>`
	Run a Docker image: Creates and runs a container.
	▪ in background
	▪ with host directory mounted into the container
	▪ with port forwarding from host to container
	▪ image name (and optional entrypoint process)

`docker run`
`-ti`
`<image> /bin/sh`
	Run a Docker image and open a shell within the container
	▪ … with forwarding of local terminal
	▪ Image name and shell (or „`/bin/bash`“)

`docker ps -a`	Prints all containers (without –a = only running containers)
`docker commit <container> qaware/foo`	Store container as local image
`docker kill <container>` `docker rm <container>`	Terminate container (send SIGKILL to entrypoint process) Remove container
`docker rmi -f <image>`	Remove local image

More commands: https://coderwall.com/p/2es5jw/docker-cheat-sheet-with-examples, https://docs.docker.com/reference

Helpful Commands for Container Troubleshooting



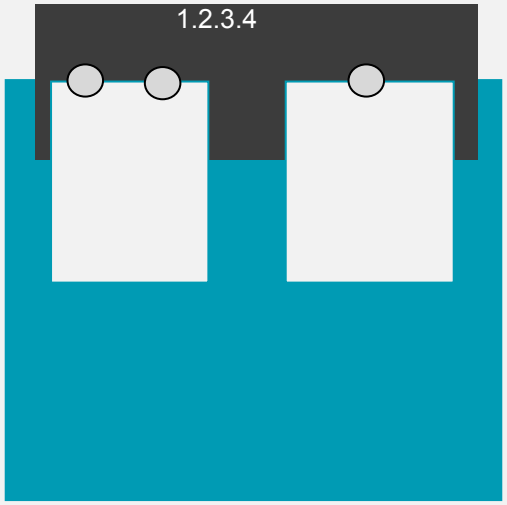
Command	Action
`docker inspect <container>`	Shows container metadata (e.g. IP)
`docker logs <container>`	Prints container syslog
`docker top <container>`	Prints all running processes within a container (like `ps -a` within the container)
`docker exec -ti <container> /bin/sh`	Connect terminal to running container
`docker stats <container>`	Shows container runtime statistics (e.g. CPU usage, IO intensity, …)

Weitere Kommandos: https://coderwall.com/p/2es5jw/docker-cheat-sheet-with-examples, https://docs.docker.com/reference

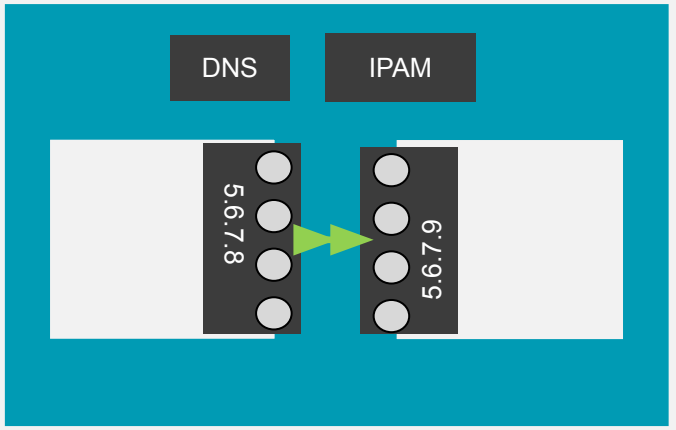
Docker Networking Modes



Bridge

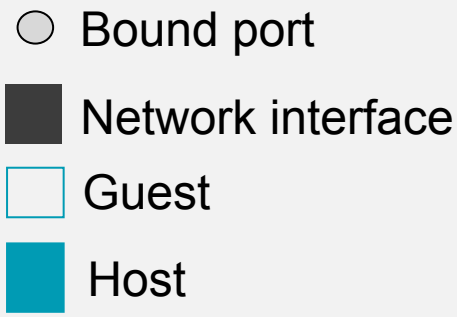


Host



Overlay Network

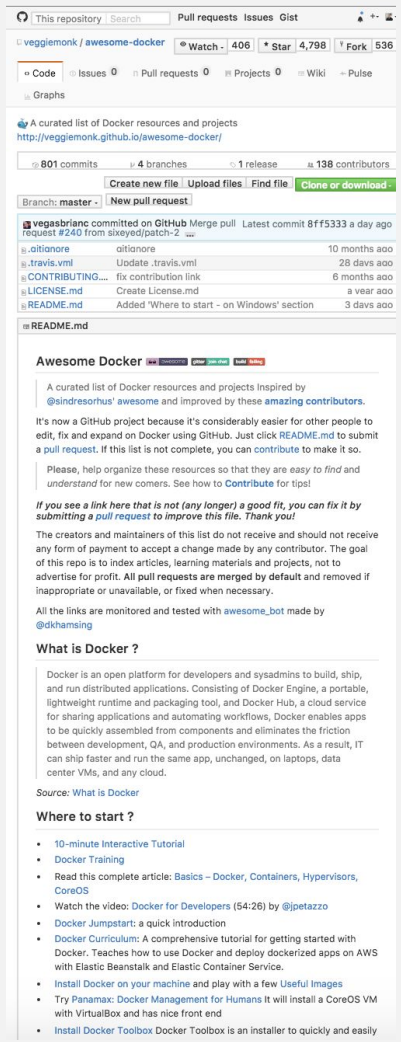
```
docker network ls
docker network inspect bridge
docker network create --driver overlay multi-host-network
docker network connect multi-host-network container1
```

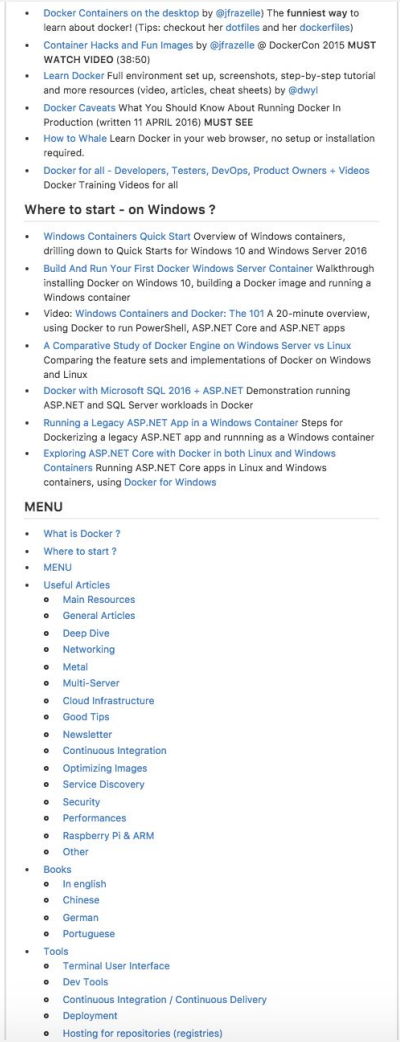


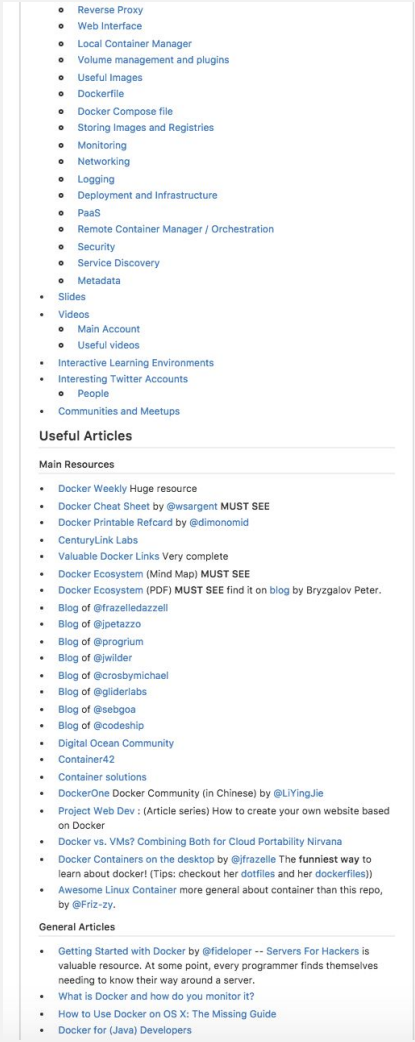


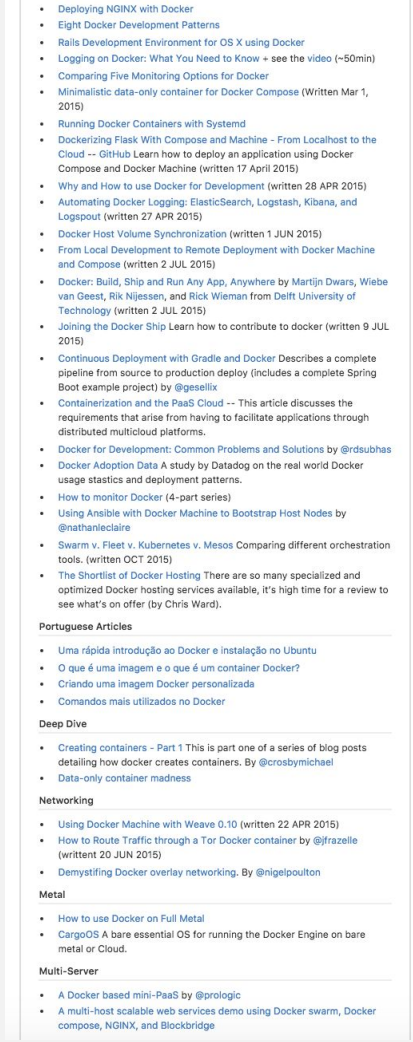
Addendum

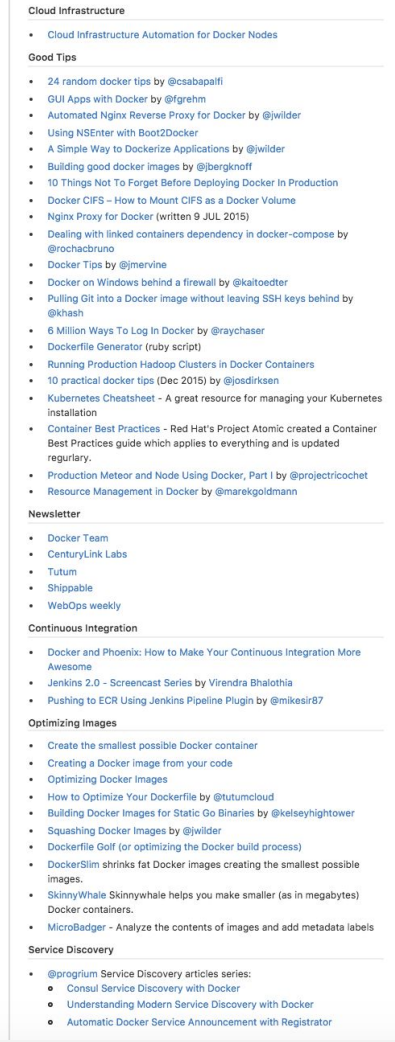
https://github.com/veggiemonk/awesome-docker











...